Theorem 1 (Rencher Theorem 10.6)**.** *The increase in R^2 due to z can be expressed as*

$$R^2_{y\boldsymbol{w}} \; - \; R^2_{y\boldsymbol{x}} \; = \; \frac{(\hat{r}_{yz} \; - \; r_{yz})^2}{1 \; - \; R^2_{zx}} \; , \tag{A375}$$

where $\hat{r}_{yz} = (\hat{\boldsymbol{\beta}}^*_{zx})^T \boldsymbol{r}_{yx}$ is a "predicted" value of r_{yz} based on the relationship of z to the $\boldsymbol{x}$'s.

The following equality shows that $\hat{r}_{yz} = (\hat{\boldsymbol{\beta}}^*_{zx})^T \boldsymbol{r}_{yx}$ is indeed a prediction of r_{yz}:

$$\begin{aligned}
\left(\hat{\boldsymbol{\beta}}^*_{zx}\right)^T \boldsymbol{r}_{yx} \; &= \; \frac{1}{s_z} \, \boldsymbol{D}_x \, \hat{\boldsymbol{\beta}}^T_{zx} \frac{1}{s_y} \, \boldsymbol{D}_x^{-1} \boldsymbol{s}_{yx} \\
&= \; \frac{1}{s_z \; s_y} \, \hat{\boldsymbol{\beta}}^T_{zx} \frac{1}{n-1} \, \sum_{i=1}^{n} (\boldsymbol{x}_i - \bar{\boldsymbol{x}})(y_i - \bar{y}) \\
&= \; \frac{1}{s_z \; s_y} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (\hat{\boldsymbol{\beta}}^T_{zx} \boldsymbol{x}_i - \hat{\boldsymbol{\beta}}^T_{zx} \bar{\boldsymbol{x}})(y_i - \bar{y}) \\
&= \; \frac{1}{s_z \; s_y} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (\hat{z}_i - \bar{\hat{z}})(y_i - \bar{y}) \\
&= \; \frac{1}{s_z \; s_y} \, \hat{s}_{yz} \; = \; \hat{r}_{yz} \; .
\end{aligned} \tag{A376}$$

If z is orthogonal to $\boldsymbol{x}$ (i.e., if $\boldsymbol{r}_{zx} = \boldsymbol{0}$), then $\hat{\boldsymbol{\beta}}^*_{zx} = \boldsymbol{0}$, which implies that $\hat{r}_{yz} = 0$ and $R^2_{zx} = 0$. In this case, Eq. (A375) can be written as

$$R^2_{y\boldsymbol{w}} \; - \; R^2_{y\boldsymbol{x}} \; = \; r^2_{yz} \; . \tag{A377}$$

Consequently, if all x_i are independent from each other, then

$$R^2_{y\boldsymbol{x}} \; = \; \sum_{j=1}^{k} r^2_{yx_j} \; . \tag{A378}$$

The contribution of z to R^2 can either be less than or greater than r_{yz}. If the correlation r_{yz} can be predicted from $\boldsymbol{x}$, then $\hat{r}_{yz}$ is close to r_{yz} and, therefore, z has contributes less to R^2 than r^2_{yz}. Next, we compute the contribution of z to R^2 explicitly. The correlation between y and z is

$$r_{yz} \; = \; \frac{1}{s_z \; s_y} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (z_i - \bar{z})(y_i - \bar{y}) \; = \; \frac{1}{s_z \; s_y} \, s_{yz} \; . \tag{A379}$$

We assume that $\bar{z} = \bar{\hat{z}}$. We want to express the information gain using the mean squared error (MSE) $1/(n-1) \sum_{i=1}^{n} (\hat{z}_i - z_i)^2$. We define the error $e_i \; := \; \hat{z}_i - z_i$ at sample i with $\bar{e} \; = \; \bar{\hat{z}} - \bar{z} \; = \; 0$. Therefore, the MSE is equal to the empirical variance $s^2_e = 1/(n-1) \sum_{i=1}^{n} e^2_i$. The correlation r_{ey} between the target y and the error e is

$$r_{ey} \; = \; \frac{1}{s_y \; s_e} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (e_i \; - \; \bar{e}) \; (y_i - \bar{y}) \; . \tag{A380}$$

Using Eq. (A376) and Eq. (A379), we can express the difference between the estimate $\hat{r}_{yz}$ and the true correlation r_{yz} by:

$$\begin{aligned}
\hat{r}_{yz} \; - \; r_{yz} \; &= \; \frac{1}{s_z \; s_y} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (\hat{z}_i - \bar{\hat{z}})(y_i - \bar{y}) \; - \; \frac{1}{s_z \; s_y} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (z_i - \bar{z})(y_i - \bar{y}) \\
&= \; \frac{1}{s_z \; s_y} \, \frac{1}{n-1} \, \sum_{i=1}^{n} (\hat{z}_i - z_i)(y_i - \bar{y}) \; .
\end{aligned} \tag{A381}$$

The information gain can now be expressed by the correlation r_{ey} between the target y and the error e:

$$R^2_{y\boldsymbol{w}} \; - \; R^2_{y\boldsymbol{x}} \; = \; \frac{(\hat{r}_{yz} \; - \; r_{yz})^2}{1 \; - \; R^2_{zx}} \; = \; \frac{\frac{1}{s_z^2 \, s_y^2} \, \frac{1}{(n-1)^2} \, \left(\sum_{i=1}^n (\hat{z}_i - z_i)(y_i - \bar{y})\right)^2}{\frac{\frac{1}{n-1} \, \sum_{i=1}^n (\hat{z}_i \; - \; z_i)^2}{\frac{1}{n-1} \, \sum_{i=1}^n (z_i \; - \; \bar{z})^2}} \tag{A382}$$

$$= \; \frac{\frac{1}{s_y^2} \, \frac{1}{(n-1)^2} \, \left(\sum_{i=1}^n (\hat{z}_i - z_i)(y_i - \bar{y})\right)^2}{\frac{1}{n-1} \, \sum_{i=1}^n \left(\hat{z}_i \; - \; z_i\right)^2}$$

$$= \; r^2_{ey} \; .$$

The information gain is the squared correlation r^2_{ey} between the target y and the error e. **The information gain is the information in z about y, which is not contained in $\boldsymbol{x}$.**

A9.2 Input Zeroing

The simplest contribution analysis method is Input Zeroing, where just an input is set to zero to determine its contribution to the output. Input Zeroing sets a particular input x_i to zero and then computes the network's output. For the original input $\boldsymbol{x} = (x_1, \ldots, x_d)$ and the input with $x_i = 0$, i.e. $\tilde{\boldsymbol{x}}_i = (x_1, \ldots, x_{i-1}, 0, x_{i+1}, \ldots, x_d)$, we compute $\Delta x_i = F(\boldsymbol{x}) \; - \; F(\tilde{\boldsymbol{x}}_i)$ to obtain the contribution of x_i. We obtain for the difference of $F(\boldsymbol{x})$ to the baseline of average zeroing $\frac{1}{d} \sum_{i=1}^d F(\tilde{\boldsymbol{x}}_i)$:

$$F(\boldsymbol{x}) \; - \; \frac{1}{d} \, \sum_{i=1}^d F(\tilde{\boldsymbol{x}}_i) \; = \; \frac{1}{d} \, \sum_{i=1}^d \Delta x_i \; . \tag{A383}$$

The problem is that the $F(\tilde{\boldsymbol{x}}_i)$ have to be computed d-times, that is, for each input component zeroed out.

Input Zeroing does not recognize redundant inputs, i.e. each one of the inputs is sufficient to produce the output but if all inputs are missing at the same time then the output changes. In contrast, Integrated Gradients (IG) and Layer-Wise Relevance Propagation (LRP) detect the relevance of an input even if it is redundant.

A9.3 Integrated Gradients

Integrated gradients is a recently introduced method [125]. Integrated gradients decomposes the difference $F(\boldsymbol{x}) - F(\tilde{\boldsymbol{x}})$ between the network output $F(\boldsymbol{x})$ and a baseline $F(\tilde{\boldsymbol{x}})$:

$$F(\boldsymbol{x}) \; - \; F(\tilde{\boldsymbol{x}}) \; = \; \sum_{i=1}^d (x_i \; - \; \tilde{x}_i) \, \int_{t=0}^1 \frac{\partial F}{\partial x_i} (\tilde{\boldsymbol{x}} + t(\boldsymbol{x} - \tilde{\boldsymbol{x}})) \, \mathrm{d}t \tag{A384}$$

$$\approx \; \sum_{i=1}^d (x_i \; - \; \tilde{x}_i) \, \frac{1}{m} \, \sum_{k=1}^m \frac{\partial F}{\partial x_i} (\tilde{\boldsymbol{x}} + (k/m)(\boldsymbol{x} - \tilde{\boldsymbol{x}})) \; .$$

In contrast to previous approaches, we have F and its derivative to evaluate only m-times, where $m < d$.

The equality can be seen if we define $\boldsymbol{h} = \boldsymbol{x} - \tilde{\boldsymbol{x}}$ and

$$\begin{cases} g : [0, 1] \to \mathbb{R} \\ g(t) = F(\boldsymbol{x} + t\boldsymbol{h}) \; . \end{cases} \tag{A385}$$

Consequently, we have

$$F(\boldsymbol{x} + \boldsymbol{h}) - F(\boldsymbol{x}) \; = \; g(1) - g(0) \; = \; \int_0^1 g'(t) \, \mathrm{d}t \tag{A386}$$

$$= \; \int_0^1 \left(\sum_{i=1}^d \frac{\partial F}{\partial x_i} (\boldsymbol{x} + t\boldsymbol{h}) \, h_i \right) \, \mathrm{d}t \; = \; \sum_{i=1}^d \, \left(\int_0^1 \frac{\partial F}{\partial x_i} (\boldsymbol{x} + t\boldsymbol{h}) \, \mathrm{d}t \right) \, h_i \; . \tag{A387}$$

For the final reward decomposition, we obtain

$$F(\boldsymbol{x}) \; = \; \sum_{i=1}^d \left((x_i \; - \; \tilde{x}_i) \, \frac{1}{m} \, \sum_{k=1}^m \frac{\partial F}{\partial x_i} (\tilde{\boldsymbol{x}} + (k/m)(\boldsymbol{x} - \tilde{\boldsymbol{x}})) \; + \; \frac{1}{d} \, F(\tilde{\boldsymbol{x}}) \right) \; . \tag{A388}$$

A9.4 Layer-Wise Relevance Propagation

Layer-Wise Relevance Propagation (LRP) [3] has been introduced to interpret machine learning models. LRP is an extension of the contribution-propagation algorithm [71] based on the contribution approach [94]. Recently "excitation backprop" was proposed [147], which is like LPR but uses only positive weights and shifts the activation function to have non-negative values. Both algorithms assign a relevance or importance value to each node of a neural network which describes how much it contributed to generating the network output. The relevance or importance is recursively propagated back: A neuron is important to the network output if it has been important to its parents, and its parents have been important to the network output. LRP moves through a neural network like backpropagation: it starts at the output, redistributes the relevance scores of one layer to the previous layer until the input layer is reached. The redistribution procedure satisfies a local relevance conservation principle. All relevance values that a node obtains from its parents will be redistributed to its children. This is analog to Kirchhoff's first law for the conservation of electric charge or the continuity equation in physics for transportation in general form. LRP has been used for deep neural networks (DNN) [84] and for recurrent neural networks like LSTM [1].

We consider a neural network with activation x_i for neuron i. The weight from neuron l to neuron i is denoted by w_{il}. The activation function is g and net_i is the netinput to neuron i with bias b_i. We have following forward propagation rules:

$$\mathrm{net}_i \;=\; \sum_l w_{il}\, x_l \;, \tag{A389}$$

$$x_i \;=\; f_i(\mathrm{net}_i) \;=\; g(\mathrm{net}_i + b_i) \;. \tag{A390}$$

Let R_i be the relevance for neuron i and $R_{i \leftarrow k}$ the share of relevance R_k that flows from neuron k in the higher layer to neuron i in the lower layer. The parameter z_{ik} is a weighting for the share of R_k of neuron k that flows to neuron i. We define $R_{i \leftarrow k}$ as

$$R_{i \leftarrow k} \;=\; \frac{z_{ik}}{\sum_l z_{lk}}\, R_k \;. \tag{A391}$$

The relative contributions z_{ik} are previously defined as [3, 84, 1]:

$$z_{ik} \;=\; w_{ik}\, x_k \;. \tag{A392}$$

Here, z_{ik} is the contribution of x_k to the netinput value net_i. If neuron k is removed from the network, then z_{ik} will be the difference to the original net_i.

The relevance R_i of neuron i is the sum of relevances it obtains from its parents k from a layer above:

$$R_i \;=\; \sum_k R_{i \leftarrow k} \;. \tag{A393}$$

Furthermore, a unit k passes on all its relevance R_k to its children, which are units i of the layer below:

$$R_k \;=\; \sum_i R_{i \leftarrow k} \;. \tag{A394}$$

It follows the *conservation of relevance*. The sum of relevances R_k of units k in a layer is equal to the sum of relevances R_i of units i of a layer below:

$$\sum_k R_k \;=\; \sum_k \sum_i R_{i \leftarrow k} \;=\; \sum_i \sum_k R_{i \leftarrow k} \;=\; \sum_i R_i \;. \tag{A395}$$

The scalar output $g(\boldsymbol{x})$ of a neural network with input $\boldsymbol{x} = (x_1, \ldots, x_d)$ is considered as relevance R which is decomposed into contributions R_i of the inputs x_i:

$$\sum_i R_i \;=\; R \;=\; g(\boldsymbol{x}) \;. \tag{A396}$$

The decomposition is valid for recurrent neural networks, where the relevance at the output is distributed across the sequence elements of the input sequence.